

## Tablas para control del flujo de los hoteles

reglas claras:

- **customers** = “cliente/billing” (quién paga / Stripe / datos fiscales)
- **debacu\_eval\_organizations** = “tenant” (el hotel/empresa dentro de Debacu Eval)
- **debacu\_eval\_org\_members** = “permisos” (usuarios que entran a ese tenant)
- **debacu\_eval\_hotel\_profile** = “configuración operativa” (ADR, estacionalidad, etc.)

```
usaremos update debacu_eval_organizations
set customer_id = '059a0113-ff49-49af-9be8-77d97a9ef866'
where id = 'b5909245-9bb5-488e-a7f2-4fb4762922a1';
```

### Guía de arquitectura y flujo Debacu Eval (Documento vivo)

#### 0) Definiciones (las que tú marcas)

- **customers** = *cliente/billing*  
Quién paga. Datos fiscales. Stripe customer/subscription. Puede pagar por 1 o varios tenants según el futuro, pero ahora 1:1 está bien.
- **debacu\_eval\_organizations** = *tenant*  
El “hotel/empresa” dentro de Debacu Eval (ámbito de datos y RLS).
- **debacu\_eval\_org\_members** = *permisos*  
Usuarios (auth.users) que pueden entrar al tenant, con **role** y **status**.
- **debacu\_eval\_hotel\_profile** = *configuración operativa*  
Parámetros operativos: checkin/checkout, ADR base, estacionalidad, etc.
- **debacu\_eval\_properties** = *hoteles / centros / unidades operativas dentro del tenant*  
Aquí es donde cuadra el multi-hotel por org sin romper el tenant.  
En demo podrás tener 1 property por org, pero dejamos soportado N.

#### 1) Regla de oro de multi-hotel (para no romper nada)

- **org\_id** = **scope de seguridad y RLS** (lo que separa tenants).
- **property\_id** = **scope operativo** (qué hotel dentro del tenant).
- Todas las tablas “de negocio” nuevas deben llevar:
  - **org\_id** NOT NULL
  - **property\_id** NULLABLE (de momento) → en producción lo haremos NOT NULL cuando el flujo esté cerrado.

En desarrollo: `property_id` puede ser null temporalmente; en nuevas escrituras intentaremos meterlo siempre desde Edge.

---

## 2) Tablas “core” y responsabilidades

### 2.1. Identidad y acceso

- `debacu_eval_organizations`
  - Crea el tenant.
- `debacu_eval_org_members`
  - Controla quién entra. OWNER/ADMIN/STAFF + ACTIVE/INVITED/ . . .
- (Opcional más adelante) `debacu_eval_customers` o tabla de unión customer ↔ org si necesitas 1 billing para varios orgs.

### 2.2. Operación hotelera

- `debacu_eval_properties`
  - Cada fila = un hotel (o propiedad) dentro del org.
- `debacu_eval_hotel_profile`
  - Config de operación (horas, reglas, etc.) **normalizada**.

### 2.3. Núcleo de datos de huéspedes (nuevo estándar)

A partir de ahora, para todo lo nuevo:

- `debacu_eval_guest_index` = “huésped” global del tenant (por `identity_key` hash)
- `debacu_eval_guest_stays` = “estancias / visitas / reservas” (una por estancia)
- `debacu_evaluations` = tu tabla legacy / histórico de incidencias (se mantiene, pero dejamos de ampliarla como eje central)

**Idea simple:**

- `guest_index` responde “¿quién es?”
  - `guest_stays` responde “¿cuándo vino y qué pasó?”
  - `evidences/incidents` responden “¿qué prueba o incidencia tengo?”
-

### 3) Flujo 1: Admin acepta solicitud de acceso (admin@debacu.com)

#### Objetivo

Cuando un hotel solicita acceso, **admin ejecuta un único flujo** que:

1. valida la solicitud
2. crea tenant y estructura mínima
3. crea permisos/membership
4. envía invitación al email del hotel para que cree password y entre

#### Artefactos que se crean (mínimo viable)

##### A) Tenant

- Insert en `debacu_eval_organizations`
  - `id`, `name`, `status=ACTIVE` (o `PENDING` si quieres), `timestamps`

##### B) Property “por defecto”

- Insert en `debacu_eval_properties`
  - `org_id`, `code (canon)`, `name`, `timezone`, `is_active=true`
  - Regla: **1 property default** al crear el org (aunque luego haya más).

##### C) Perfil operativo

- Insert en `debacu_eval_hotel_profile`
  - `org_id`, `property_id` (si aplica), valores por defecto
  - Importante: `check-in/out` en formato fijo (te lo normalizo cuando me pases la Edge que falla)

##### D) Permisos

- Insert en `debacu_eval_org_members`
  - `org_id`, `auth_user_id` (del invitado si existe ya, si no por `invited_email`)
  - `role=OWNER` (para el hotel)
  - `status=INVITED`

##### E) Invitación

- Invitar por Supabase Auth Admin:
  - `auth.admin.inviteUserByEmail(email, { redirectTo: ... })`
  - En metadata puedes meter: `org_id`, `invited_role`, `property_id_default`

## F) (Opcional) Enlace con billing

- Si ya tienes customers (billing) creado antes o durante la solicitud:
    - guarda `customer_id` o `creator_customer_id` en org o en una tabla de mapping.
- 

## 4) Contratos de Edge Functions (normas)

Para que no se des controle:

### 4.1. Respuesta estándar

Siempre:

```
{ "ok": true, "data": ... }  
{ "ok": false, "error": { "code": "...", "message": "...", "details": ... } }
```

### 4.2. Seguridad

- Todas las Edge Functions: **JWT-only** (requireUser)
- Para admin: comprobación dura:
  - o bien `email == admin@debacu.com`
  - o un rol “ADMIN\_SYSTEM” en una tabla de admins (mejor, pero en demo vale email hardcodeado)

### 4.3. Scope

- Cualquier insert/consulta de tenant requiere `org_id`.
  - Cualquier operación operativa requiere `property_id` (si ya existe selector) o se usa el default.
- 

## 5) Plan de trabajo (cómo seguimos sin perdernos)

Vamos pantalla por pantalla.

**Paso A (ahora):** Admin aceptación acceso

- Me pasas:
  1. Screenshot de la pantalla admin
  2. Edge Function que usa esa pantalla (la primera)
  3. Tablas implicadas (si no las sé, con nombres basta)

Y yo te devuelvo:

- Flujo cerrado (SQL + Edge)

- Qué columnas faltan / sobran
- Qué RLS/políticas hay que tener para que funcione
- Qué queda “pendiente” para producción

**Paso B:** Hotel invitado crea contraseña y entra

- Pantalla activate / finalize + Edge(s)

**Paso C:** Perfil del hotel (hotel\_profile) + normalización inputs

- Aquí resolvemos lo de check-in/out y “todo en mayúsculas” (lo hacemos en Edge o en DB con trigger; en demo mejor en Edge).

**Paso D:** Primeras pantallas de negocio (guest\_index / guest\_stays)

- Todo lo nuevo ya va a esas tablas.

## 6) Nota importante sobre tus datos demo actuales (para que no te explote)

- **No conviertas a NOT NULL property\_id ahora.**
- En demo puedes dejar debacu\_evaluations como histórico “sucio”.
- A partir del momento que creemos hotel1@ y hotel2@:
  - todo registro nuevo debe entrar ya por guest\_index y guest\_stays con org\_id y property\_id correctos.

Una vez aquí seguimos con

**Documento Operativo: Flujo Admin → Org + Property + Permisos + Invitación (DEBACU\_EVAL)**

## 0) Contexto y principio rector

- **Frontend (React) NO toca tablas críticas;** todo lo sensible pasa por **Edge Functions** con JWT y validaciones.

DEBACU – Arquitectura y Estado ...

- **Modelo multi-tenant estricto:** cualquier lectura/escritura de datos de negocio debe estar **acotada por org\_id** (o customer\_id ligado a esa org).

DEBACU – Arquitectura y Estado ...

- **Planes/Suscripción:** “source of truth” en BD (no preguntar a Stripe para decidir permisos).

Seguimos con el modelo definiti...

- **Idempotencia obligatoria** en eventos de Stripe/webhooks y en flujos de provisioning (evitar duplicar org/members/customer).
- 

## 1) Modelo validado (mínimo necesario)

### 1.1 Entidades y relaciones

#### A) customers (facturación / cuenta comercial)

- `customers.id` (PK)
- datos fiscales, email, `app_id`, `stripe_customer_id`, etc.
- Este es el “billing account”.

#### B) debacu\_eval\_organizations (tenant / hotel “lógico”)

- `debacu_eval_organizations.id` (PK)
- `customer_id` (FK → `customers.id`) ☒ ya lo tienes y ya está relleno (`orgs_without_customer=0`)
- campos: name, cif, address, `property_type`, `rooms_count`, etc.

#### C) debacu\_eval\_properties (hotel “físico” / multipropiedad)

- Recomendación: **separar “org” de “property”** desde ya, aunque al principio 1 org = 1 property.
- `properties.id` (PK), `org_id` (FK), nombre comercial, dirección, timezone, checkin/out defaults, etc.
- Esto te desbloquea multipropiedad sin rehacer nada después.

#### D) debacu\_eval\_org\_members (usuarios del tenant)

- `org_id`, `user_id`, `role`, `status` (ACTIVE/INVITED/SUSPENDED...)
- **Seats** se cuentan aquí (por org). Esto encaja perfecto con el enfoque por tenant.

#### E) subscriptions / plans

- Suscripción vigente por (`customer_id`, `app_id`) con índice parcial “current” (no solo ACTIVE).
- `plans.max_users` es la fuente de verdad de seats por plan.  
migración `debacu_eval_subscript...`

#### F) debacu\_eval\_access\_requests (pipeline admin)

- registro de solicitudes “PENDING → APPROVED/REJECTED”

- guarda `customer_id`, `org_id`, auditoría de email enviado y `reviewed_by/at`.
- 

## 2) Flujo exacto (Admin)

### 2.1 Estado actual (lo que ya tienes)

- UI Admin (`AdminSolicitudesAccesoPage`) llama Edge: `LIST`, `APPROVE`, `REJECT`, `RESEND`.
- Edge actual (`debacu-eval-admin-access-requests`) hace:
  - `getOrCreate customer`
  - `upsert profile`
  - `ensureOrganization`
  - `ensureDefaultImportProfile`
  - `ensureFreeTrialSubscription` (o equivalente)
  - `invite/recovery email` por Supabase
  - `ensureOwnerInvitedMembership`
  - `update access_request status + audit del email`

Eso está bien como idea.

### 2.2 Flujo definitivo “APPROVE” (orden exacto y por qué)

#### Entrada (body)

- `requestId`
- `reviewedBy`
- `decisionNotes`
- `sendEmail`
- `siteUrl` (para construir redirect estable)

#### Paso 1 — Lock lógico (idempotencia de aprobación)

- Leer `access_request`.
- Si `status != PENDING` → devolver 200 con lo que ya tiene (`customer_id/org_id`) y NO repetir efectos.
- Esto evita doble click / retries.

#### Paso 2 — Provisioning “core” (idempotente)

- (2.1) **Customer:** `getOrCreate` por email (idempotente).

- (2.2) **Organization:** ensureOrganization y (crítico) **siempre set customer\_id** (ya lo corrigiste en BD, pero el flujo debe garantizarlo).
- (2.3) **Property:** ensureDefaultProperty (si no existe).
  - Si hoy no existe tabla, créala ya. Si no quieres aún, al menos deja “placeholder” en el doc: *en cuanto haya multipropiedad, property será obligatoria*.
- (2.4) **Defaults operativos:** import\_profile por defecto, settings iniciales, etc.

### Paso 3 — Suscripción (source of truth BD)

- Crear TRIAL/ FREE según tu política.
- OJO: ahora mismo tu vista devuelve `subscription_status=null` para una org → eso significa “sin habilitación”. Ese caso debe ser válido pero con acceso bloqueado a módulos premium (o a todo salvo onboarding).

### Paso 4 — Membership (asignación owner INVITED)

- Upsert member por `invited_email` (idempotente).
- Estado correcto en esta fase: INVITED.
- El paso que cambia a ACTIVE ocurre en **invite\_finalize** cuando el usuario ya tiene sesión JWT y se valida.

### Paso 5 — Email

- `inviteUserByEmail` con `redirectTo = /auth/activate?org_id=...`
- Si “already registered” → recovery email (tu fallback está bien).
- Auditar `last_email_status/at/detail`.

### Paso 6 — Cierre

- Update request a APPROVED, set `customer_id`, `org_id`, audit fields.

Este orden minimiza “huérfanos”: primero aseguras core (customer/org/property) y luego invitas.

## 3) Qué inserts deben ser “transaccionales (todo o nada)”

Aquí voy directo y sin azúcar:

### 3.1 Realidad técnica

Con supabase - js contra PostgREST **NO** tienes transacciones multi-step reales (BEGIN/COMMIT) dentro de Edge Functions.

Si exiges “todo o nada” *de verdad*, tienes solo 2 opciones:

#### Opción A (recomendada si quieres atomicidad real):

Crear **una única función SQL** (RPC)



`admin_approve_access_request(request_id, ...)` que haga todo en una transacción.

Sí: es “RPC”. Pero es la única forma limpia de atomicidad en Postgres sin conexión directa. (No hay magia.)

### **Opción B (si mantienes “0 RPC” a rajatabla):**

Haces una “transacción de negocio” tipo **SAGA**:

- cada paso es **idempotente** (upsert/ensure)
- si algo falla, lo puedes **reanudar** sin duplicar ni corromper
- el sistema converge a un estado consistente

Tu Edge actual ya está encaminada a **Opción B**. Si te empeñas en “todo o nada” literal, te tocará permitir al menos 1 RPC para onboarding admin.

## **3.2 Mi decisión recomendada (práctica)**

- En **desarrollo** y **producción**: usa **SAGA idempotente** (Opción B), pero **declara explícitamente** qué es “core” y qué es “best effort”.

### **Core (debe quedar sí o sí):**

- customers (getOrCreate)
- organizations (ensure + customer\_id)
- properties (ensure default) — si existe tabla
- org\_members OWNER INVITED

### **Best effort (puede fallar sin romper integridad):**

- email
- import\_profile default
- trial subscription (si falla, se puede reintentar; pero ideal que quede)

Para “best effort”, guardas error en `access_request` y permites **RESEND** / **REPAIR**.

---

## **4) RLS: aislamiento tenant (sin fugas)**

### **4.1 Regla simple: toda tabla de negocio debe estar “org-scoped”**

- Si una tabla pertenece al tenant: **tiene org\_id**.
- Las políticas usan:
  - `exists (select 1 from debacu_eval_org_members m where m.org_id = row.org_id and m.user_id = auth.uid() and m.status='ACTIVE')`

## 4.2 Qué tablas deben ser RLS “hard”

- `debacu_eval_organizations` (select/update limitado a miembros ACTIVE de esa org)
- `debacu_eval_properties`
- `debacu_eval_org_members` (select solo miembros de tu org; insert/update solo OWNER/ADMIN)
- `debacu_eval_access_requests` (solo admin; hotel no debe leer esto)
- tablas de negocio (evaluations, imports, exports...) siempre por `org_id`

## 4.3 Vista `debacu_eval_org_entitlements_v`

- Bien que exista (no PII).
  - Aun así:
    - o la dejas solo accesible por `service_role` (Edge)
    - o si la quieres exponer al cliente: RLS con `org_members` ACTIVE
- 

# 5) Contrato listo para `hotel1@` y `hotel2@`

## 5.1 Contrato de “APPROVE”

Respuesta mínima estable (lo que tu UI puede asumir):

```
{
  "ok": true,
  "requestId": "uuid",
  "customerId": "uuid",
  "orgId": "uuid",
  "propertyId": "uuid|null",
  "subscription": {
    "status": "ACTIVE|TRIAL_ACTIVE|PENDING_PAYMENT|null",
    "planCode": "FREE|BASIC|MEDIUM|PREMIUM|null",
    "maxUsers": 1,
    "seatsUsed": 0
  },
  "invite": {
    "sendEmail": true,
    "emailSent": true,
    "detail": "INVITE (...)|RECOVERY|...",
    "redirectTo": "https://.../auth/activate?org_id=..."
  }
}
```

## 5.2 Flujo `hotel1@` / `hotel2@`

- `hotel1@` aprobado → org + property + owner invited → email → activa password → finalize → member ACTIVE
- `hotel2@` igual

- **Seat check:** antes de activar un segundo usuario, verificar `seatsUsed < maxUsers`.
- 

## 6) Ajustes concretos que yo haría YA (según tu estado real)

1. **No dependas del caso “customer\_id = org\_id”** (lo tienes en dev). En prod eso te puede liar debugging y migraciones. Deja permitido en dev, pero el flujo debe generar IDs correctos siempre.
  2. **Tu vista ahora muestra:**
    - hotel las rosas: `subscription_status=null` y `plan_code=null` → por tanto, si el usuario entra, debes bloquear módulos (o forzar trial). Si tu producto exige trial automático, entonces el APPROVE debe crear trial siempre.
  3. **Seats:** hotel las rosas tiene `seats_used=0` → significa que todavía no has insertado el OWNER member para esa org (o no hay usuario creado). Tu flujo APPROVE ya crea member INVITED por email: perfecto; cuando el usuario complete `invite_finalize` pasará a ACTIVE.
- 

## 7) Check-list de “sólido” (cuando terminemos este hilo)

- APPROVE idempotente (si ya aprobado, no duplica efectos)
  - Org siempre tiene `customer_id` real
  - Default property creada (o decisión explícita de no usar aún)
  - Membership OWNER INVITED creado siempre
  - Trial/subscription creada según política
  - Entitlements endpoint/view devuelve `plan + seatsUsed + maxUsers`
  - Seat enforcement en: `invite_finalize` / `add_member` / cualquier alta usuario
  - RLS org-scoped en todas las tablas tenant
- 

## Documentos base (referencias internas del proyecto)

- Modelo de suscripciones / events / seats / helper:  
Seguimos con el modelo definiti...
- Arquitectura y principios (Edge Functions, no RPC, RLS, multi-tenant):

- Estado actual de tablas/políticas/índices (subscriptions/plans/events):  
migración debacu\_eval\_subscript...
- 

## Lo siguiente (ya, sin vueltas)

1. **Revisar documento:** ya está arriba (este es el doc operativo).
2. **Validar modelo:** customers / organizations / org\_members / properties (te falta definir/crear properties si aún no existe).
3. **Diseñar el flujo exacto de la Edge Function de admin:** ya está especificado por pasos.
4. **Definir transaccionalidad:** o aceptas SAGA idempotente (recomendado) o introduces 1 RPC para atomicidad real.
5. **Ajustar RLS:** org-scoped en todo, access\_requests solo admin.
6. **Contrato hotel1@ / hotel2@:** definido.

Si quieres que lo dejemos “cerrado” de verdad en el próximo mensaje, pégame **solo**:

- el esquema (columns) de debacu\_eval\_properties si ya existe (o dime “no existe”),  
y
- el código de tu requireAdmin (para confirmar cómo decides quién es admin).

Con eso te devuelvo:

- la Edge Function **final** (lista para copiar/pegar) con el flujo definitivo,
- y las políticas RLS exactas para organizations / properties / org\_members / view.

Perfecto: con la captura ya sabemos **que debacu\_eval\_properties existe** y cuál es su schema real. Y con tu RequireAdmin vemos el patrón “admin\_whoami + cache por usuario” (correcto).

Voy directo a dejar esto sólido en 3 frentes:

1. **Modelo (org → properties)**
  2. **Edge Function APPROVE (añadir property default + idempotencia)**
  3. **RLS de debacu\_eval\_properties (y lo mínimo que debe cumplir para no mezclar tenants)**
-

# 1) Validación rápida del modelo con tu tabla `debacu_eval_properties`

Por la captura, columnas:

- `id` `uuid` (PK)
- `org_id` `uuid` (NO NULL)
- `code` `text` (NO NULL)
- `name` `text` (NO NULL)
- `category` `smallint` (nullable)
- `address/city/country/timezone` `text` (nullable)
- `is_active` `boolean` (NO NULL)
- `created_at` `timestampz` (NO NULL)
- `updated_at` `timestampz` (NO NULL)

**Bien:** ya tienes la separación tenant (`org`) vs asset físico (`property`).

**Riesgo actual (si no lo has puesto):** si no existe constraint/índice, puedes terminar con properties duplicadas por org (ej: 2 “MAIN” activos). Solución: índice único por (`org_id`, `code`) como mínimo.

---

## 2) Edge Function: añadir `ensureDefaultProperty()` y que sea idempotente

Tu APPROVE hoy crea: `customer` → `org` → `import_profile` → `trial` → `invite/recovery` → `member` → `update request`.

Le falta: **property default**.

### 2.1 Regla práctica (sin líos)

- Cada org debe tener **1 property “MAIN”** (`code` fijo, `name` = `company_name`).
- Si ya existe, no insertes otra.
- `is_active=true`.
- `country` en mayúsculas (ya lo haces en org), `timezone` por defecto: “Europe/Madrid” si viene vacío.

### 2.2 Implementación lista para copiar (añádela en la misma Edge)

```
async function ensureDefaultProperty(  
  sbAdmin: ReturnType<typeof supabaseServiceClient>,  
  params: {
```

```

    org_id: string;
    name: string;
    address?: string | null;
    city?: string | null;
    country?: string | null;
    timezone?: string | null;
    category?: number | null;
  },
) {
  const org_id = safeStr(params.org_id);
  if (!org_id) throw new Error("missing_org_id_for_property");
  const DEFAULT_CODE = "MAIN";
  // 1) ¿ya existe?
  const { data: existing, error: findErr } = await sbAdmin
    .from("debacu_eval_properties")
    .select("id, code, is_active")
    .eq("org_id", org_id)
    .eq("code", DEFAULT_CODE)
    .limit(1)
    .maybeSingle();
  if (findErr) throw new Error(`db_properties_find_failed:${findErr.message}`);
  if (existing?.id) {
    // opcional: garantizar activo
    if (existing.is_active === false) {
      const { error: updErr } = await sbAdmin
        .from("debacu_eval_properties")
        .update({ is_active: true, updated_at: new Date().toISOString() })
        .eq("id", existing.id);
      if (updErr) throw new Error(`db_properties_reactivate_failed:${updErr.message}`);
    }
    return { created: false, id: existing.id as string, code: DEFAULT_CODE };
  }
  // 2) insert idempotente por código fijo
  const now = new Date().toISOString();
  const payload: any = {
    id: crypto.randomUUID(),
    org_id,
    code: DEFAULT_CODE,
    name: safeStr(params.name) || `Property ${DEFAULT_CODE}`,
    category: typeof params.category === "number" ? params.category : null,
    address: params.address ?? null,
    city: params.city ?? null,
    country: safeUpper(params.country ?? "ESP") || "ESP",
    timezone: safeStr(params.timezone) || "Europe/Madrid",
    is_active: true,
    created_at: now,
    updated_at: now,
  };
  const { data: inserted, error: insErr } = await sbAdmin
    .from("debacu_eval_properties")
    .insert(payload)
    .select("id")
    .single();
  if (insErr) throw new Error(`db_properties_insert_failed:${insErr.message}`);
  return { created: true, id: inserted.id as string, code: DEFAULT_CODE };
}

```

## 2.3 Dónde llamarla en tu APPROVE

Justo después de `ensureOrganization(...)` (porque ya tienes `org_id`) y antes del email:

```

const orgRes = await ensureOrganization(...);
const propertyRes = await ensureDefaultProperty(sbAdmin, {

```

```
org_id: orgRes.org_id,  
name: company_name || `Hotel ${email}`,  
address,  
city,  
country,  
timezone: "Europe/Madrid", // o request.timezone si lo tienes  
category: null, // o mapear property_type → category si quieres  
});
```

Y añádelo al response:

```
property: {  
  created: propertyRes.created,  
  id: propertyRes.id,  
  code: propertyRes.code,  
},
```

---

### 3) “Todo o nada” (transaccionalidad) con tu enfoque actual

Te lo digo claro:

- Con PostgREST + supabase - js **no hay transacción SQL real multi-step**.
- Por tanto, tu “todo o nada” debe ser **consistencia por idempotencia** (SAGA), que es lo que ya estás haciendo.

**Qué debe ser “core” (sí o sí)**

- customer
- org
- property MAIN
- member OWNER (INVITED)

**Qué puede fallar sin romperlo**

- email (invite/recovery)
- trial (si falla, lo puedes reparar)

Tu RESEND ya es la parte “reparar”. Para trial, podrías añadir acción REPAIR\_BILLING más adelante si hace falta.

---

### 4) RLS para debacu\_eval\_properties (aislamiento tenant)

Objetivo: nadie ve properties de otras orgs.

#### 4.1 Índices/constraints recomendados (imprescindible)

En SQL editor:

```
-- 1) único por org + code
create unique index if not exists debacu_eval_properties_org_code_ux
on public.debacu_eval_properties (org_id, code);
-- 2) filtro por org (performance RLS)
create index if not exists debacu_eval_properties_org_id_ix
on public.debacu_eval_properties (org_id);
```

## 4.2 Activar RLS

```
alter table public.debacu_eval_properties enable row level security;
```

## 4.3 Políticas (mínimas y seguras)

**SELECT:** solo miembros ACTIVE de esa org

```
create policy "properties_select_active_members"
on public.debacu_eval_properties
for select
to authenticated
using (
    exists (
        select 1
        from public.debacu_eval_org_members m
        where m.org_id = debacu_eval_properties.org_id
            and m.user_id = auth.uid()
            and m.status = 'ACTIVE'
    )
);
```

**INSERT:** solo OWNER/ADMIN ACTIVE (y service\_role para Edge)

Ojo: tu provisioning lo hace service\_role, así que esto lo cubre.

```
create policy "properties_insert_owner_admin"
on public.debacu_eval_properties
for insert
to authenticated
with check (
    exists (
        select 1
        from public.debacu_eval_org_members m
        where m.org_id = debacu_eval_properties.org_id
            and m.user_id = auth.uid()
            and m.status = 'ACTIVE'
            and m.role in ('OWNER', 'ADMIN')
    )
);
create policy "properties_insert_service_role"
on public.debacu_eval_properties
for insert
to service_role
with check (true);
```

**UPDATE:** igual que insert

```
create policy "properties_update_owner_admin"
on public.debacu_eval_properties
for update
to authenticated
using (
    exists (
        select 1
        from public.debacu_eval_org_members m
        where m.org_id = debacu_eval_properties.org_id
            and m.user_id = auth.uid()
            and m.status = 'ACTIVE'
    )
);
```



```

        and m.role in ('OWNER', 'ADMIN')
    )
)
with check (
    exists (
        select 1
        from public.debacu_eval_org_members m
        where m.org_id = debacu_eval_properties.org_id
            and m.user_id = auth.uid()
            and m.status = 'ACTIVE'
            and m.role in ('OWNER', 'ADMIN')
    )
);

```

**DELETE** (si lo permites): normalmente NO. Si lo necesitas, solo OWNER. Si no, no crees policy de delete y listo.

---

## 5) RequireAdmin: lo que tienes está bien, pero NO es seguridad

Tu RequireAdmin frontend está correcto (cache por usuario id y no “heredar admin”). Pero recuerda:

- **Esto NO protege nada** por sí solo.
- La seguridad real está en `requireAdmin(req)` de Edge + RLS.

Si `admin_whoami()` consulta a una tabla tipo `debacu_eval_admin_users` o valida email allowlist, perfecto. Lo importante es que `requireAdmin` en Edge sea **la fuente de verdad**.

---

## 6) Contrato listo para hotel1@ y hotel2@

Con property ya creada, tu APPROVE debería devolver SIEMPRE:

- `customer_id`
- `org_id`
- `property_id` (MAIN)
- `invite_redirect_to` con `org_id`

Esto permite:

- hotel1@ → activar → finalize → member ACTIVE en org1
  - hotel2@ → activar → finalize → member ACTIVE en org2
  - y las queries posteriores se filtran por `org_id` sin fugas.
-

## **PROMPT CONTINUIDAD — Multi-tenant sólido + RLS + Property MAIN + pruebas reales**

Seguimos con Debacu Evaluation.

Estado actual confirmado:

- Edge `debacu-eval-admin-access-requests` ya ajustada y funcionando.
- `admin_whoami` funcionando.
- Flujo APPROVE crea:
  - `customer`
  - `organization`
  - `property MAIN`
  - `org_member OWNER (INVITED)`
  - `free trial`
  - `email invite/recovery`
- Tabla `debacu_eval_access_requests` actualizada correctamente.
- Tabla `debacu_eval_properties` ya existe.
- Multi-tenant real por `org_id`.

Ahora quiero **cerrar correctamente la arquitectura multi-tenant a nivel de seguridad y consistencia**.

Objetivo de esta sesión:

1. Auditar y ajustar RLS en:
  - `debacu_eval_organizations`
  - `debacu_eval_org_members`
  - `debacu_eval_properties`
  - `debacu_eval_import_profiles`
  - `debacu_eval_subscriptions`
  - `customers` (si aplica)
  - `debacu_eval_access_requests` (solo admin)
2. Definir reglas claras:
  - Usuario autenticado solo puede ver datos de su `org_id`.
  - ADMIN (`admin@debacu.com`) puede ver todo.
  - Service role puede escribir todo.

- Un usuario puede pertenecer a varias orgs.
- El acceso depende de:  
status = 'ACTIVE'  
role in ('OWNER','ADMIN','STAFF')

### 3. Validar caso real:

- hotel1@debacu.com → org A
- hotel2@debacu.com → org B
- Mismo email no puede contaminar org distinta.
- property MAIN debe existir solo una vez por org.
- No debe poder verse ninguna property de otra org.

### 4. Confirmar diseño correcto para escalar a multipropiedad:

- 1 org → N properties
- Todas las tablas futuras deben referenciar:  
org\_id  
property\_id (cuando aplique)
- No depender solo de customer\_id.

Quiero:

- Propuesta concreta de RLS por tabla (policies SQL exactas).
- Índices recomendados (performance + unicidad).
- Validación conceptual del modelo multi-tenant.
- Lista de posibles agujeros de seguridad típicos en Supabase multi-tenant.
- Checklist final antes de pasar a módulo Revenue Intelligence.

No quiero teoría general.

Quiero:

- SQL concreto
- Reglas claras
- Arquitectura limpia
- Sin mezclar mundo admin con mundo hotel